Name: Yash Gawade
Roll No: 46

Import Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt
```

Create Dataset

```
data = {
    'Age': [22, 25, 47, 52, 46, 56, 23, 40, 60, 36],
    'Gender': ['Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'Income': [25000, 30000, 50000, 60000, 52000, 65000, 27000, 48000, 70000, 45000],
    'Purchased': ['No', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)
df
```

	Age	Gender	Income	Purchased
0	22	Female	25000	No
1	25	Male	30000	No
2	47	Female	50000	Yes
3	52	Male	60000	Yes
4	46	Female	52000	Yes

Next steps: Generate code with df New interactive sheet

Data Preprocessing (Encode Categorical Data)

```
# Encode Gender
le_gender = LabelEncoder()
df['Gender'] = le_gender.fit_transform(df['Gender'])

# Encode Purchased (Target Variable)
le_purchase = LabelEncoder()
df['Purchased'] = le_purchase.fit_transform(df['Purchased'])

df
```

Define Features (X) and Target (y)

```
X = df[['Age', 'Gender', 'Income']]
y = df['Purchased']
```

Split Data into Train & Test

```
X_train, X_test, y_train, y_test = train_test_split(
    X,y, test_size =0.3, random_state=42
)
```

Feature Scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Apply Logistic Regression

```
model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

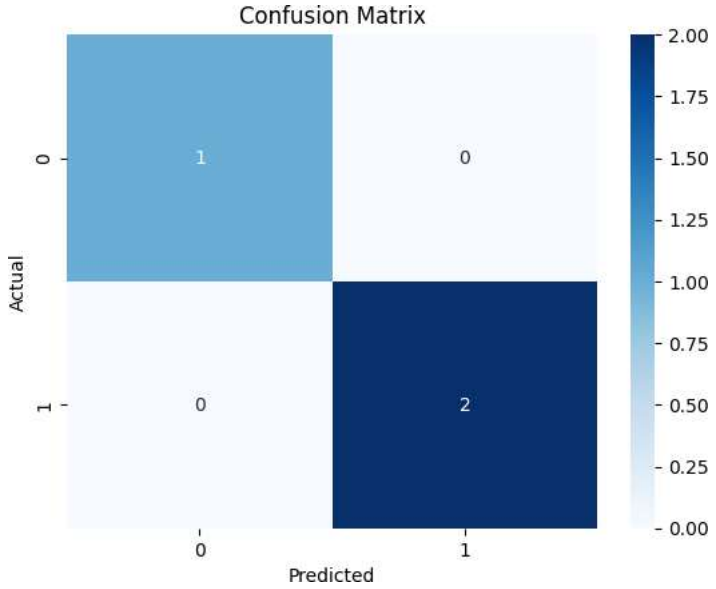
```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[1 0]
 [0 2]]
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Calculate Performance Metrics

Evaluation Metrics

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

COSMETICS SHOP DATASET MODEL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
```

```
data = {
    'Age': [22, 25, 47, 52, 46, 56, 23, 40, 60, 36],
    'Gender': ['Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male'],
    'Income': [25000, 30000, 50000, 60000, 52000, 65000, 27000, 48000, 70000, 45000],
    'Purchased': ['No', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)
```

```
le_gender = LabelEncoder()
df['Gender'] = le_gender.fit_transform(df['Gender'])

le_purchase = LabelEncoder()
df['Purchased'] = le_purchase.fit_transform(df['Purchased'])
```

Feature and Target

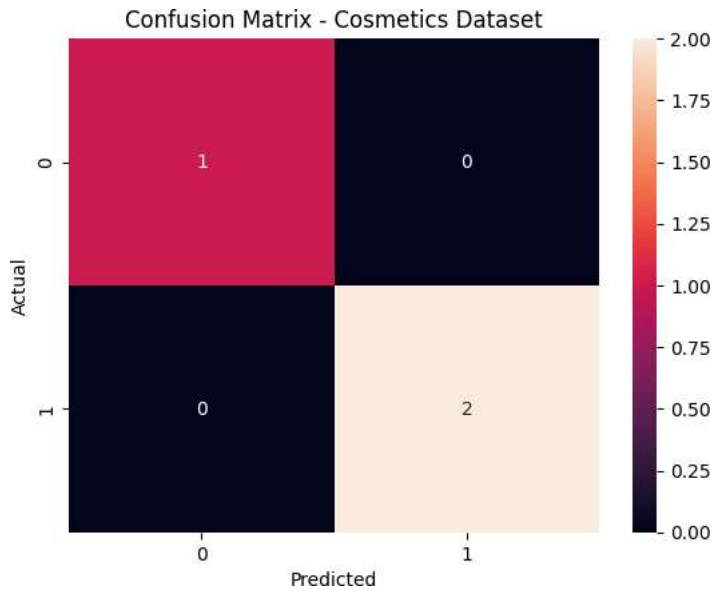
```
X = df[['Age', 'Gender', 'Income']]
y = df['Purchased']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model2 = LogisticRegression()
model2.fit(X_train, y_train)

y_pred = model2.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Cosmetics Dataset")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Evaluation Metrics

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

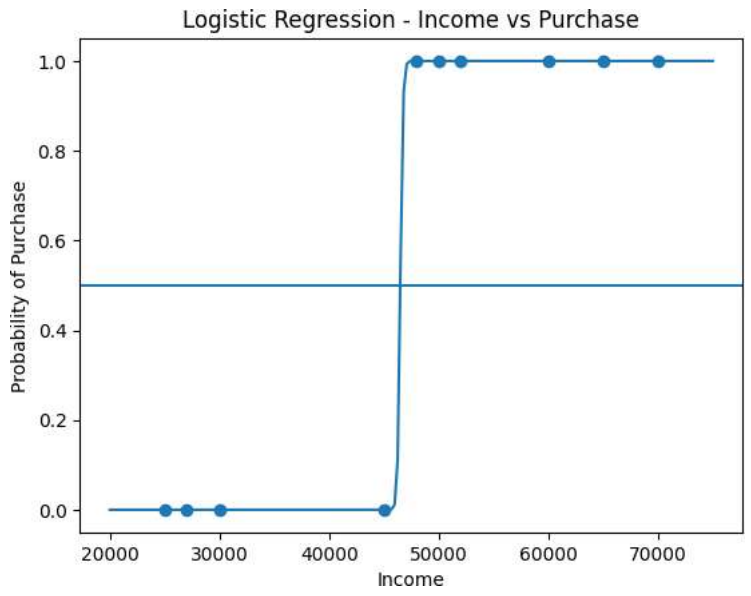
```
X_income = df[['Income']].values
y_purchase = df['Purchased'].values
```

```
model_sigmoid = LogisticRegression()
model_sigmoid.fit(X_income, y_purchase)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
X_test_income = np.linspace(df['Income'].min()-5000,
                            df['Income'].max()+5000,
                           200).reshape(-1, 1)

y_prob_income = model_sigmoid.predict_proba(X_test_income)[:, 1]
```

```
plt.figure()
plt.scatter(X_income, y_purchase)
plt.plot(X_test_income, y_prob_income)
plt.axhline(0.5)
plt.title("Logistic Regression - Income vs Purchase")
plt.xlabel("Income")
plt.ylabel("Probability of Purchase")
plt.show()
```




```
b0 = model.intercept_[0]
b1 = model.coef_[0][0]
decision_boundary = -b0 / b1

print("Decision Boundary (Spending Score):", decision_boundary)
```

```
Decision Boundary (Spending Score): -14734782.799653452
```